

Legal Ease

Document Generation Logic in routers.py

All document generation is handled by one FastAPI router mounted at **/documents**. Requests are validated immediately, then rendered in a background task so the API stays fast and the client polls for status.

Endpoints

Method	Path	Purpose
POST	/documents/generate	Validate fields, create a pending record, queue generation. Returns 202.
GET	/documents/{id}	Check status (pending, ready, failed) and get the download URL.
GET	/documents/{id}/download	Stream the finished PDF. Returns 409 if not ready.

Generation Flow

- **Validate:** `templates.missing_fields` checks required placeholders and returns a 422 listing any that are missing.
- **Record:** a pending row is created and tied to the authenticated user.
- **Queue:** `_run_generation` runs as a background task.
- **Build:** render template text, optionally add a plain-language summary, convert to PDF, save to storage.
- **Finish:** status becomes ready (or failed with an error message).
- **Serve:** the download route checks ownership and status before returning the file.

Design Notes

- Ownership checks return 404 (not 403) so document IDs are never leaked across users.
- Service modules (`templates`, `analysis`, `renderer`, `storage`) hold the logic; the router only orchestrates.
- For production, swap `BackgroundTasks` for a durable queue (Celery, RQ, or Arq) so jobs survive restarts.
- Generated documents are template-based drafts, not legal advice; keep a disclaimer in the output.

Source: routers.py

```
"""routers.py - Document generation routes for Legal Ease (FastAPI)."""
from __future__ import annotations

import uuid
from enum import Enum
from typing import Any

from fastapi import APIRouter, BackgroundTasks, Depends, HTTPException, status
from fastapi.responses import FileResponse
from pydantic import BaseModel, Field

from .auth import User, get_current_user
from .db import documents_repo
from .services import analysis, renderer, storage, templates
```

```

router = APIRouter(prefix="/documents", tags=["documents"])

class DocType(str, Enum):
    NDA = "nda"
    LEASE = "lease"
    SERVICE_AGREEMENT = "service_agreement"

class DocStatus(str, Enum):
    PENDING = "pending"
    READY = "ready"
    FAILED = "failed"

class GenerateRequest(BaseModel):
    doc_type: DocType
    fields: dict[str, Any] = Field(..., description="Template placeholder values")
    include_plain_summary: bool = True

class DocumentOut(BaseModel):
    id: str
    doc_type: DocType
    status: DocStatus
    download_url: str | None = None
    error: str | None = None

def _to_out(doc: dict) -> DocumentOut:
    ready = doc["status"] == DocStatus.READY
    return DocumentOut(
        id=doc["id"],
        doc_type=doc["doc_type"],
        status=doc["status"],
        download_url=f"/documents/{doc['id']}/download" if ready else None,
        error=doc.get("error"),
    )

def _get_owned(doc_id: str, user: User) -> dict:
    """Fetch a document and enforce per-user isolation."""
    doc = documents_repo.get(doc_id)
    if doc is None or doc["user_id"] != user.id:
        raise HTTPException(status.HTTP_404_NOT_FOUND, "Document not found")
    return doc

def _run_generation(doc_id: str, req: GenerateRequest) -> None:
    """Background job: render, summarize, build PDF, store, update status."""
    try:
        text = templates.render(req.doc_type.value, req.fields)
        summary = analysis.plain_summary(text) if req.include_plain_summary else None
        pdf_bytes = renderer.to_pdf(text, summary=summary)
        path = storage.save(doc_id, pdf_bytes)
        documents_repo.update(doc_id, status=DocStatus.READY, file_path=path)
    except Exception as exc: # keep the API responsive; surface via status
        documents_repo.update(doc_id, status=DocStatus.FAILED, error=str(exc))

@router.post("/generate", response_model=DocumentOut,
             status_code=status.HTTP_202_ACCEPTED)
def generate_document(
    req: GenerateRequest,
    background: BackgroundTasks,
    user: User = Depends(get_current_user),
) -> DocumentOut:

```

```

# Validate synchronously so the client gets a fast, clear 422.
missing = templates.missing_fields(req.doc_type.value, req.fields)
if missing:
    raise HTTPException(
        status.HTTP_422_UNPROCESSABLE_ENTITY,
        detail={"missing_fields": missing},
    )

doc_id = str(uuid.uuid4())
doc = documents_repo.create(
    id=doc_id, user_id=user.id, doc_type=req.doc_type,
    status=DocStatus.PENDING,
)
background.add_task(_run_generation, doc_id, req)
return _to_out(doc)

@router.get("/{doc_id}", response_model=DocumentOut)
def get_document_status(doc_id: str, user: User = Depends(get_current_user)):
    return _to_out(_get_owned(doc_id, user))

@router.get("/{doc_id}/download")
def download_document(doc_id: str, user: User = Depends(get_current_user)):
    doc = _get_owned(doc_id, user)
    if doc["status"] != DocStatus.READY:
        raise HTTPException(status.HTTP_409_CONFLICT, "Document is not ready")
    return FileResponse(
        doc["file_path"],
        media_type="application/pdf",
        filename=f"{doc['doc_type'].value}.pdf",
    )

```